

```

# db.py
# Connects to the SAME Supabase project the
# frontend already uses
# (see .env: SUPABASE_URL /
# SUPABASE_PUBLISHABLE_KEY).
#
# NOTE: the publishable/anon key works fine
# for reads if Row Level Security
# (RLS) policies allow it. For inserts
# (e.g. logging predictions) you may
# need a service_role key stored ONLY as a
# backend environment variable –
# never commit that one to GitHub.

import os
from supabase import create_client, Client

SUPABASE_URL =
os.environ.get("SUPABASE_URL", "")
SUPABASE_KEY =
os.environ.get("SUPABASE_KEY", "") # anon
key for reads

_client: Client | None = None

def get_client() -> Client:
    global _client
    if _client is None:
        if not SUPABASE_URL or not

```

```

SUPABASE_KEY:
    raise RuntimeError(
        "SUPABASE_URL and
SUPABASE_KEY must be set as environment
variables."
    )
    _client =
create_client(SUPABASE_URL, SUPABASE_KEY)
    return _client

def fetch_trains_for_route(source: str,
destination: str) -> list[dict]:
    """
    Mirrors the matching logic already in
the frontend's search.service.ts
(fetchTrainsForRoute), so behaviour
stays consistent when we switch
USE_FASTAPI on.
    """
    client = get_client()
    resp = (
        client.table("trains")
        .select(
            "train_number, train_name,
source_code, destination_code, "
            "departure_time, arrival_time,
duration"
        )
        .execute()
    )
    rows = resp.data or []

    src = source.strip().lower()

```

```
dst = destination.strip().lower()

matches = [
    r
    for r in rows
        if (r["source_code"].lower() == src
or src in r["source_code"].lower())
        and (r["destination_code"].lower()
== dst or dst in
r["destination_code"].lower())
]
return matches if matches else rows[:5]
```